

Microservices Architecture Implementation Guide

Hands-on Lab Exercise: Spring Boot 3, Spring Cloud Discovery Server, & API Gateway

1. Architectural Theory & Context

Monolithic vs. Microservices in Enterprise Banking

In traditional banking environments, modern suites often begin as monolithic platforms where core elements—including Savings Accounts, Loans, Insurance, Forex Services, Cards, and Portfolio Tracking—reside within a single deployable application bundle (EAR/WAR). While initially simpler to maintain, this design presents severe vulnerabilities during periods of high computational stress (such as festival season shopping or automated flash transactions).

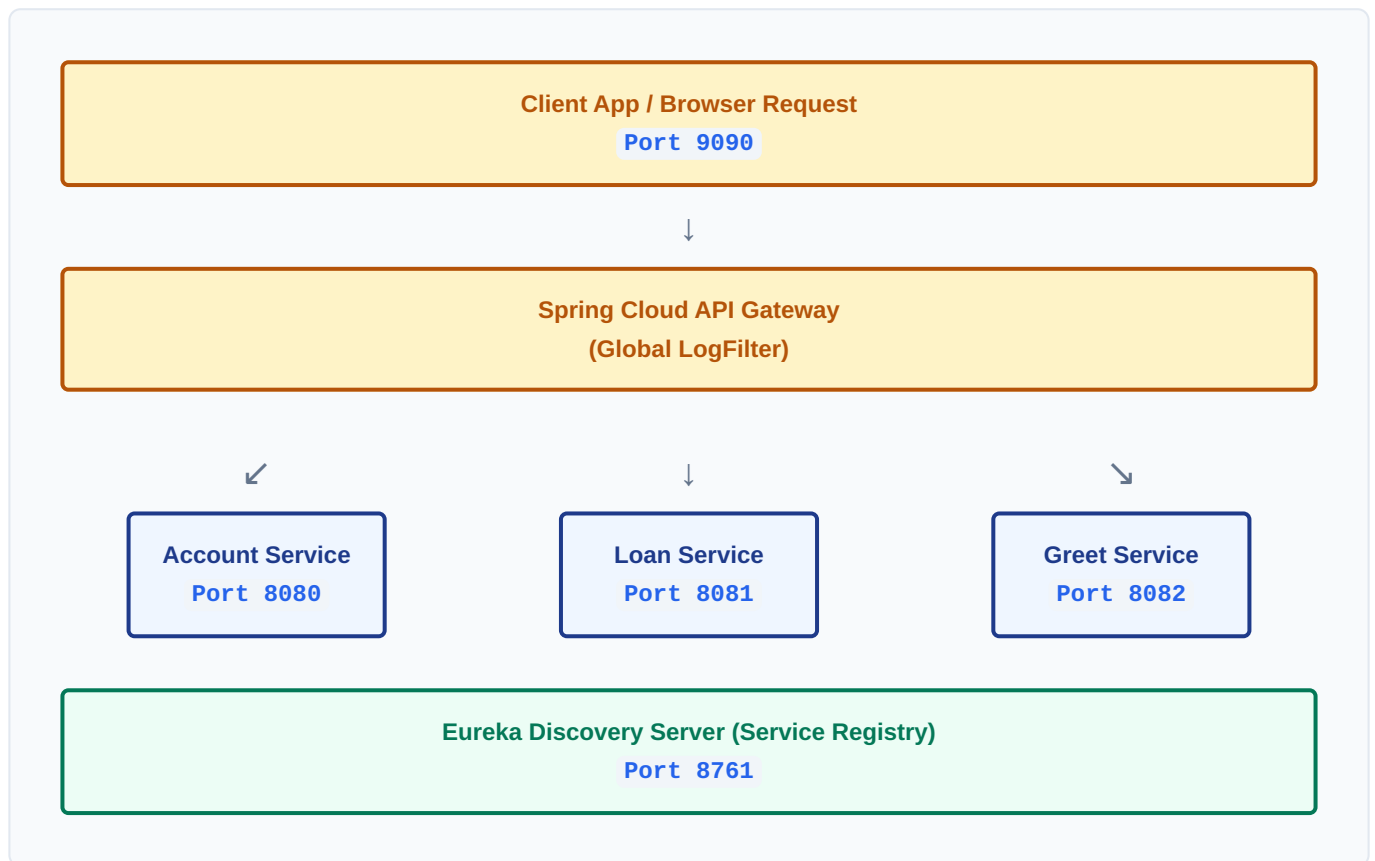
For instance, a minor memory leak within a high-traffic operation like `getAccountBalance` can rapidly consume the shared application container memory pool. Under a monolithic architecture, this memory exhaustion completely paralyzes entirely independent modules, leaving agents unable to submit loan configurations or process time-sensitive credit card locks.

The Solution: Microservices Architecture

By decomposing the application into isolated, highly domain-focused structural contexts, failures remain strictly contained. If the account service fails or throttles under extreme load, downstream services like the loan module continue to process client queries securely.

Logical Routing & Discovery Design

Moving to a distributed microservices environment introduces operational complexity: service instances deploy on dynamic, changing network addresses. To resolve this, a **Service Discovery Hub (Eureka Server)** is used to act as an automated coordinator, while an **Edge API Gateway** acts as a single point of entry to efficiently proxy and monitor traffic flows.



2. Module 1: Eureka Discovery Server

The Discovery Server maintains a live, dynamic registry of all active microservice instances. Client components query this hub to establish communication paths without hardcoding target network positions.

`pom.xml` Configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>
  <groupId>com.cognizant</groupId>
  <artifactId>eureka-discovery-server</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>eureka-discovery-server</name>

  <properties>
    <java.version>17</java.version>
    <spring-cloud.version>2023.0.1</spring-cloud.version>
  </properties>
```

```

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope font-style="italic">runtime</scope>
    <optional>true</optional>
  </dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
</project>

```

src/main/resources/application.properties

```

server.port=8761

# Prohibit the server from attempting to register with itself
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

# Mute internal telemetry noise in the log console
logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF

```

Main Application Bootstrapper

```

package com.cognizant.eurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer // Activates the Netflix Eureka Server infrastructure
public class EurekaDiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaDiscoveryServerApplication.class, args);
    }
}

```

3. Module 2: Core Banking Microservices

These downstream domain modules act as isolated functional units. They use lightweight Java 17 `record` data representations and announce their active locations directly to the centralized Eureka Server registry.

A. Account Microservice (`account-service`)

Handles operations regarding core balance lookups. Configured to run by default on `port 8080`.

`src/main/resources/application.properties`

```
server.port=8080
spring.application.name=account-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Data Payload Record & REST Controller

```
package com.cognizant.account.model;

// Java 17 immutable record definition for clean serialization
public record AccountResponse(String number, String type, double balance) {}
```

```
package com.cognizant.account.controller;

import com.cognizant.account.model.AccountResponse;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public AccountResponse getAccountDetails(@PathVariable String number) {
        // Simulates domain logic retrieval safely without external store complexity
        return new AccountResponse(number, "savings", 234343.0);
    }
}
```

B. Loan Microservice (`loan-service`)

Manages calculations for credit and auto financing metrics. To prevent host network allocation context blockages when executing on a local workstation, its port must be explicitly overridden.

Port Collision Warning

If you run multiple Spring Boot applications locally without changing the `server.port` property, the launch will fail with a `java.net.BindException: Address already in use` error. Always assign distinct ports (e.g., 8080, 8081, 8082) for each service.

`src/main/resources/application.properties`

```
server.port=8081
spring.application.name=loan-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Data Payload Record & REST Controller

```
package com.cognizant.loan.model;

public record LoanResponse(String number, String type, double loan, double emi, int tenure) {}
```

```
package com.cognizant.loan.controller;

import com.cognizant.loan.model.LoanResponse;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public LoanResponse getLoanDetails(@PathVariable String number) {
        return new LoanResponse(number, "car", 400000.0, 3258.0, 18);
    }
}
```

4. Module 3: Validation Service & Edge API Gateway

The API Gateway functions as the reverse proxy entryway. It intercepts public client requests, queries Eureka for safe instances, applies reactive routing filters, and securely forwards traffic downstream.

A. Greet Validation Service (`greet-service`)

A lightweight microservice instance running on `port 8082`, utilized to validate gateway connectivity mappings across lowercase path configurations.

```
package com.cognizant.greet.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetController {

    @GetMapping("/greet")
    public String getGreeting() {
        return "Hello World!!";
    }
}
```

B. API Gateway Engine Configuration (`api-gateway`)

Operating on `port 9090`, this component coordinates automated route mappings using structured YAML configurations.

`src/main/resources/application.yml`

```
server:
  port: 9090

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
      routes:
        - id: account_route
          uri: lb://account-service
          predicates:
            - Path=/accounts/**
        - id: loan_route
          uri: lb://loan-service
          predicates:
            - Path=/loans/**
        - id: greet_route
          uri: lb://greet-service
          predicates:
            - Path=/greet/**

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

C. Global Interceptor Request Logger

This filter class captures incoming metrics before passing operations to downstream targets, providing deep runtime visibility across the distributed ecosystem.

```
package com.cognizant.gateway.filter;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

@Component
public class LogFilter implements GlobalFilter, Ordered {

    private static final Logger logger = LoggerFactory.getLogger(LogFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        // Intercept and trace active request execution paths
        logger.info("Gateway intercepted incoming request path: {}",
            exchange.getRequest().getPath());

        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            logger.info("Gateway post-route operation status code: {}",
                exchange.getResponse().getStatusCode());
        }));
    }

    @Override
    public int getOrder() {
        // Ensures immediate pipeline tracking at the edge
        return Ordered.HIGHEST_PRECEDENCE;
    }
}
```

5. Run Sequence & Verification Steps

To ensure stable network discovery mappings, launch the distributed cluster exactly in the following sequence:

1. **Start the Eureka Registry Hub:** Execute the `eureka-discovery-server`. Confirm deployment by accessing the dashboard engine in your browser at `http://localhost:8761`. Ensure that the active instance tracking list is empty.
2. **Launch Downstream Services:** Start the `account-service`, `loan-service`, and `greet-service`. Refresh your Eureka dashboard to verify that all three application designations show up correctly as active targets.
3. **Launch the Routing Gateway:** Run the `api-gateway` engine process.

Verification Validation Commands

Execute the following queries via any standard client or browser console tool to verify that your API Gateway routes traffic and intercepts paths correctly:

```
# Test Account Application Route Routing
GET http://localhost:9090/accounts/00987987973432

# Expected Response Payload Output:
# { "number": "00987987973432", "type": "savings", "balance": 234343.0 }

# Test Loan Application Route Routing
GET http://localhost:9090/loans/H00987987972342

# Expected Response Payload Output:
# { "number": "H00987987972342", "type": "car", "loan": 400000.0, "emi": 3258.0, "tenure": 18 }

# Test Greeting Application Route Routing
GET http://localhost:9090/greet

# Expected Response Payload Output:
# Hello World!!
```

Open the `api-gateway` process console to verify that the `LogFilter` intercepts requests correctly. You should see incoming trace statements reflecting your query operations:

```
INFO c.c.gateway.filter.LogFilter - Gateway intercepted incoming request path: /accounts/
00987987973432
INFO c.c.gateway.filter.LogFilter - Gateway post-route operation status code: 200 OK
```